Cryptocurrency Wallet Tracker

CIS 344 - Group Project Report

Fall 2026

Professor: Yanilda Peralta Ramos

Group Members: Jorge Grullon , Elaine Yamilex Brea, Clifford Amoah

1. Introduction and Project Overview

The Cryptocurrency Wallet Tracker is a web-based database application developed using PHP, MySQL, HTML, and CSS. The purpose of the project is to allow users to organize cryptocurrency wallets, record transactions, and monitor their cryptocurrency holdings through an accessible website.

The application provides account registration and authentication, wallet management, transaction recording, and automatic portfolio balance calculations.

This project demonstrates how relational database concepts can be implemented in a working application through SQL queries, primary and foreign keys, table relationships, prepared statements, and CRUD operations.

The application is intended for educational purposes. It does not connect to real cryptocurrency exchanges or execute actual cryptocurrency trades.

2. Project Plan and Objectives

Our primary objective was to design and implement a cryptocurrency tracking application that integrates a MySQL database with a PHP website.

The project was organized into several development stages:

1. Select the Cryptocurrency Wallet Tracker project theme.

2. Identify entities, attributes, and database relationships.

3. Develop an EER diagram using MySQL Workbench.

4. Implement the database tables and SQL queries.

5. Connect MySQL to the PHP application.

6. Develop registration, login, wallet, and transaction functionality.

7. Implement portfolio calculations and security validation.

8. Test the application and document the results.

9. Organize the project using GitHub and Trello.

The project scope focuses on manually recorded transactions, per-user wallet management, and database-driven portfolio calculations.

3. Technologies Used

Technology | Purpose

Technology	**Purpose**
PHP	Server-side processing and application logic
MySQL	Relational database management
MySQL Workbench	EER diagram creation and SQL development
HTML	Website structure and forms
CSS	Styling, navigation, and responsive layouts
XAMPP	Local Apache development environment
Visual Studio Code	Code development and editing
Git and GitHub	Version control and source code management
Trello	Project management and group task tracking

4. Database Design

Our database is named crypto_wallet_tracker and consists of four main tables.

4.1 Users

The users table stores the information required for account registration and authentication.

Its attributes include **user_id, first_name, last_name,** email, and **password_hash**.

The user_id field is the primary key. The email field has a UNIQUE constraint to prevent duplicate email registrations.

4.2 Wallets

The wallets table stores cryptocurrency wallets created by registered users.

Its attributes include **wallet_id, user_id, wallet_name**, and **date_created**.

The wallet_id field is the primary key, and user_id is a foreign key referencing the users table.

4.3 Cryptocurrencies

The cryptocurrencies table stores the available cryptocurrency types.

Its attributes include **crypto_id, crypto_name**, and **symbol**.

The crypto_id field is the primary key, and the symbol is unique.

The database includes Bitcoin, Ethereum, Solana, Cardano, and Dogecoin.

4.4 Wallet Transactions

The wallet_transactions table records cryptocurrency activity.

Its attributes include **transaction_id, wallet_id, crypto_id**, **transaction_type, quantity, price_usd**, and **transaction_date**.

The primary key is transaction_id. The foreign keys wallet_id and crypto_id connect transactions with their respective wallets and cryptocurrencies.

Transactions can represent BUY, SELL, DEPOSIT, and WITHDRAWAL operations.

4.5 Relationships

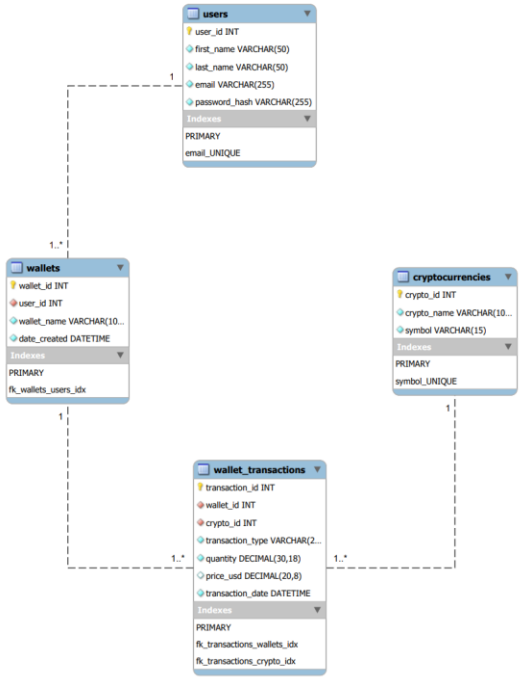
The relational database implements three one-to-many relationships:

- One user can own multiple wallets.

- One wallet can contain multiple transactions.

- One cryptocurrency can be associated with multiple transactions.

These relationships allow information to be separated into related tables instead of unnecessarily duplicating data.



5. SQL Implementation and Database Queries

SQL is a major component of our project because it retrieves, connects, and summarizes information stored across the four tables.

5.1 INNER JOIN

An INNER JOIN connects related records from two or more tables.

For example, our wallet management page connects users and wallets using the matching user_id.

Selected SQL statements include:

```
SELECT *

FROM users AS u

INNER JOIN wallets AS w

   ON u.user_id = w.user_id

WHERE u.user_id =?;
```

This retrieves wallets belonging to the currently authenticated user.

```sql
SELECT *
FROM users AS u
INNER JOIN wallets AS w
    ON u.user_id = w.user_id
WHERE u.user_id = 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	first_name	last_name	email	password_hash	wallet_id	user_id	wallet_name	date_created
3	Jorge	Grullon	jorgeagrullon@gmail.com	$2y$10$LvosIj4etYhKXbLM2kZefeoqY0dAnGfKX...	4	3	Bitcoin Savings	2026-10-18 16:09:18
3	Jorge	Grullon	jorgeagrullon@gmail.com	$2y$10$LvosIj4etYhKXbLM2kZefeoqY0dAnGfKX...	5	3	My DOGE Wallet	2026-10-18 16:23:27

5.2 Multiple-Table JOIN

Our Transaction History page joins four tables: users, wallets, wallet_transactions, and cryptocurrencies.

These relationships make it possible to display a transaction's owner, wallet name, cryptocurrency, transaction type, quantity, price, and date in one table.

Instead of storing duplicated names in the transactions table, the application retrieves related values using foreign keys.

```php
// Retrieve only the logged-in user's transactions
$sql = "
    SELECT
        t.transaction_id,
        u.first_name,
        u.last_name,
        w.wallet_name,
        c.crypto_name,
        c.symbol,
        t.transaction_type,
        t.quantity,
        t.price_usd,
        t.transaction_date
    FROM users AS u
    INNER JOIN wallets AS w
        ON u.user_id = w.user_id
    INNER JOIN wallet_transactions AS t
        ON w.wallet_id = t.wallet_id
    INNER JOIN cryptocurrencies AS c
        ON t.crypto_id = c.crypto_id
    WHERE u.user_id = ?
    ORDER BY t.transaction_id
";
```

5.3 LEFT JOIN

We also implemented LEFT JOIN queries during database testing to retrieve records that may not have related entries.

For example, a LEFT JOIN can display cryptocurrencies with their transaction counts, including cryptocurrencies that have not yet appeared in a transaction.

This demonstrates the difference between an INNER JOIN, which returns matching records, and a LEFT JOIN, which preserves records from the left table.

```sql
SELECT
    u.user_id,
    u.first_name,
    u.last_name,
    w.wallet_name
FROM users AS u
LEFT JOIN wallets AS w
    ON u.user_id = w.user_id;
```

Result Grid | Filter Rows: | Export:

user_id	first_name	last_name	wallet_name
1	John	Smith	Main Wallet
1	John	Smith	Savings Wallet
2	Maria	Lopez	Personal Wallet
3	Jorge	Grullon	Bitcoin Savings
3	Jorge	Grullon	My DOGE Wallet
4	Alex	Johnson	Alex Test Wallet

5.4 GROUP BY and Aggregate Functions

Our portfolio summary calculates cryptocurrency holdings using SUM(), CASE, and GROUP BY.

The calculation adds quantities for BUY and DEPOSIT transactions and subtracts quantities for SELL and WITHDRAWAL transactions.

The central expression is:

SUM(CASE WHEN transaction_type IN ('BUY', 'DEPOSIT') THEN quantity WHEN transaction_type IN ('SELL', 'WITHDRAWAL') THEN -quantity ELSE 0 END)

Results are grouped by user, wallet, and cryptocurrency.

This allows the application to calculate portfolio balances directly from recorded transactions rather than storing redundant balance values.

```sql
SUM(
    CASE
        WHEN t.transaction_type IN ('BUY', 'DEPOSIT')
            THEN t.quantity
        WHEN t.transaction_type IN ('SELL', 'WITHDRAWAL')
            THEN -t.quantity
        ELSE 0
    END
) AS total_quantity
```

```sql
GROUP BY
    u.user_id,
    u.first_name,
    u.last_name,
    w.wallet_id,
    w.wallet_name,
    c.crypto_id,
    c.crypto_name,
    c.symbol
```

6. PHP Application Development

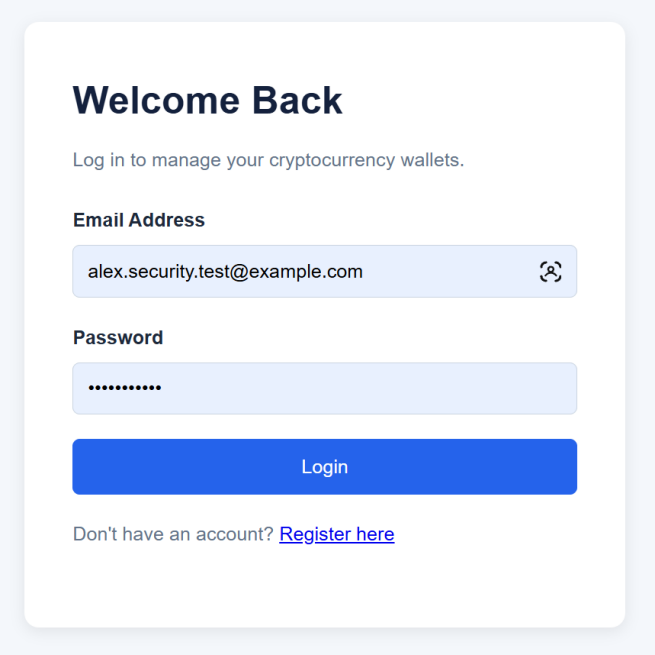
6.1 User Registration and Login

The application provides registration and login forms.

During registration, PHP validates submitted information and hashes the password using password_hash() before storing the account in MySQL.

During login, the application retrieves the user's password hash and checks the submitted password using password_verify().

PHP sessions maintain authentication across protected pages.

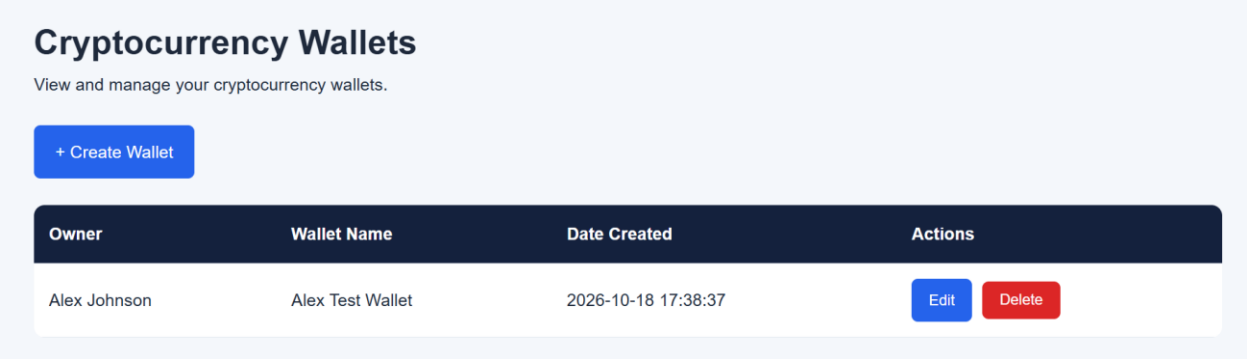


6.2 Wallet Management

Authenticated users can create, view, rename, and delete their wallets through the website.

PHP processes form submissions and uses SQL statements to insert, retrieve, update, and delete wallet records.

Each wallet belongs to a specific registered user.



6.3 Transaction Management

The Transaction History page displays cryptocurrency transactions associated with the logged-in user's wallets.

Users can record purchases, sales, deposits, and withdrawals. They can also edit and delete transactions through the corresponding actions.

Each transaction records a cryptocurrency, quantity, transaction type, date, and optional or required price depending on the transaction type.

6.4 Portfolio Summary

The Portfolio Summary page retrieves wallet transactions and calculates current cryptocurrency quantities.

The calculations use SQL aggregate functions and JOIN operations.

For example, if a wallet records a purchase of 0.05 BTC and a sale of 0.01 BTC, the calculated remaining quantity is 0.04 BTC.

This feature demonstrates how SQL can transform transaction records into meaningful summary information.

7. Security and Validation

We implemented several security measures throughout the application.

Authentication: Protected pages require an authenticated PHP session.

Password Protection: Passwords are hashed before being saved and verified during login.

Prepared Statements: Parameterized SQL queries are used for database operations involving user-supplied input.

Input Validation: The application checks fields such as wallet names, transaction quantities, transaction types, and prices.

User Ownership: Database operations are restricted to records associated with the authenticated user's account.

Form Protection: Security tokens help validate sensitive form submissions.

Balance Validation: The application rejects sales and withdrawals that exceed available cryptocurrency holdings.

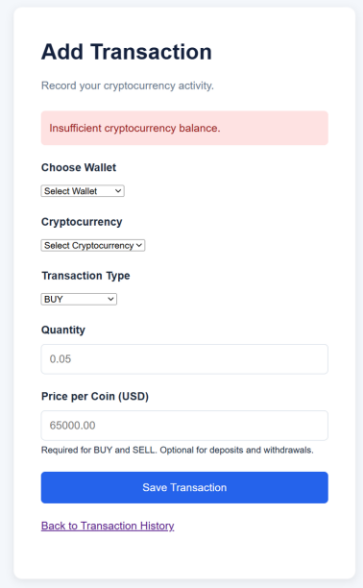
These measures improve the security and reliability of the application, although a comprehensive production security audit is outside the scope of this classroom project.

8. Testing and Results

We tested the application using the local PHP and MySQL environment.

Test	**Observed Result**
Account registration	Account creation and hashed password storage demonstrated
Login and logout	Authentication and session behavior demonstrated
Protected page access	Seven tested pages redirected logged-out users to Login
Wallet ownership	Another account's wallet edit URL displayed "Wallet not found."
Transaction ownership	Another account's transaction edit URL displayed "Transaction not found."
Excessive SELL quantity	Application displayed an insufficient-balance error
Excessive WITHDRAWAL quantity	Application displayed an insufficient-balance error
Portfolio calculation	Balances were displayed using transaction aggregation
SQL JOIN queries	Related records were successfully retrieved

These results demonstrate that the tested core features operate as expected.



Wallet not found.

9. Group Collaboration and Project Management

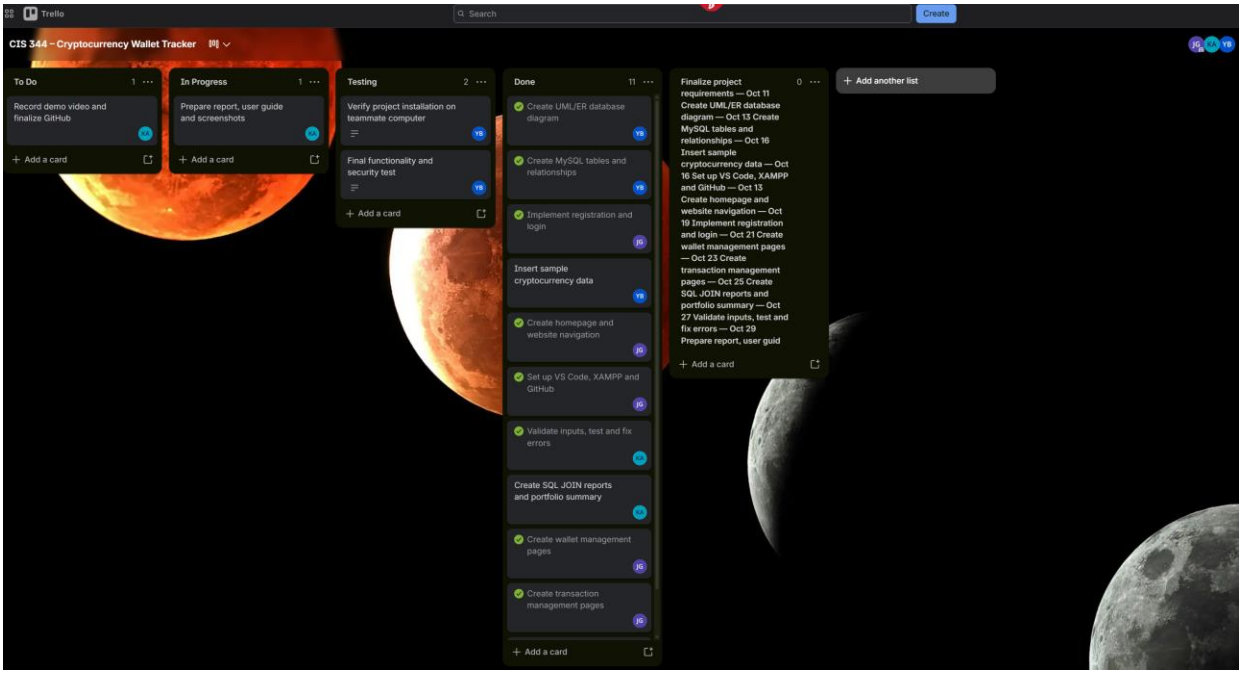
Our team used GitHub to maintain the application source code and Trello to organize project tasks.

The Trello board uses four categories: To Do, In Progress, Testing, and Done.

GitHub contains the PHP application, CSS stylesheet, SQL schema, starter data, EER diagram, and README installation instructions.

Group Contributions

Each group member's contributions will be finalized using their actual completed tasks, reviews, documentation, and testing activities.



10. Limitations and Future Improvements

The Cryptocurrency Wallet Tracker is a simulated tracking application and does not execute cryptocurrency purchases or sales.

It does not retrieve live market prices or connect directly to external cryptocurrency wallets.

Potential future improvements include live cryptocurrency pricing, portfolio value charts, transaction searching, additional account management features, and deployment to an online hosting environment.

These improvements are not necessary for the current database-focused educational scope.

11. Conclusion

The Cryptocurrency Wallet Tracker demonstrates the integration of MySQL relational database design with PHP backend processing and a web-based user interface.

Through this project, we applied database concepts including primary keys, foreign keys, relationships, SQL JOIN operations, aggregate functions, CRUD functionality, and database validation.

The completed application allows registered users to manage simulated cryptocurrency wallets, record transactions, and review calculated portfolio balances.

The project provided practical experience with database-driven web development, application testing, version control, and collaborative project management.

12. Repository and Project Resources

GitHub Repository:

https://github.com/jorgeg365/Cryptocurrency-Wallet-Tracker

Trello Project Management Board:

https://trello.com/b/pccRZeHd/cis-344-cryptocurrency-wallet-tracker

Demo Link:

https://drive.google.com/file/d/1UIEJ-iw1u8aDvqTUv8lazzttUXoPIotq/view?usp=sharing